

ReOS

Revival Operating System

PRESENTATION SCRIPT

A spoken script to explain the system - a short version and an in-depth, screen-by-screen walkthrough.

An AI revenue-recovery control plane for Razorpay merchants.

Razorpay AI Buildathon - Track 03: AI Revenue Recovery

by Ajay John Paul - github.com/Kajayjohnpauldev/REOS

Live demo: <http://localhost:3000> - Login: ajay@lumina.example / 12345619

Part 1 - The short script

~90 seconds. The elevator pitch. Say it in one breath, confidently.

Hi, I'm Ajay - and this is ReOS, the Revival Operating System.

Every Razorpay merchant quietly loses revenue in four places: failed payments, abandoned checkouts, failed subscriptions, and overdue invoices. It's fragmented, and most of it never gets worked.

The tools that do exist are risky. Blind retries can double-charge a customer. Dunning messages lean on dark patterns. And crucially - nobody can prove what they actually recovered versus what would have happened anyway.

ReOS is a revenue-recovery control plane. For every at-risk event it does three things.

One - it reconciles against live state before it acts. So if a 'failed' webhook lands on a payment that's already captured, ReOS re-reads reality and turns it into a logged no-op - never a second charge.

Two - it takes exactly one bounded action, inside a policy the merchant owns: a retry, a consented reminder, a human escalation, or a stop.

Three - it records recovered money in an append-only ledger and subtracts refunds back out, so every rupee is provable.

The AI drafts, classifies, and explains - but it never moves money. Only the ledger does. Manipulative messages are refused, and every refusal is logged.

There's a live Replay studio that walks six real scenarios and measures incremental recovery against a holdout; role-based dashboards for the whole team; and Baymax, an in-app AI assistant on Google Gemini.

It runs fully offline with zero keys, and the credibility core is tested.

That's ReOS: recover more revenue, and prove every rupee.

Part 2 - The in-depth script

~6-8 minutes. A live, screen-by-screen walkthrough. Stage directions are in italics.

0. Framing

[Open <http://localhost:3000>]

"Let me walk you through ReOS end to end - not slides, the actual working system."

"The thesis is one sentence: given a verified revenue-risk event, recover the most money with the least customer friction - and prove exactly what happened, why, and when we chose to stop."

1. Login and roles

[Sign in with [ajay@lumina.example / 12345619 \(Admin\)](#)]

"ReOS is multi-user and role-based. There are four roles - Admin, Recovery Operator, Finance Analyst, and Auditor - and each one gets a dashboard and permissions tailored to their job. I'll sign in as the Admin."

"Notice it's a real login with hashed passwords and sessions - not a mock screen."

2. Overview - the daily cockpit

[Land on Overview]

"This is the operator's home. Four KPIs across the top: at-risk value - about Rs 1.95 lakh; gross recovered - Rs 38.7K; incremental recovered - Rs 35.7K; and net recovered."

"Hover any KPI and it reveals a mini-graph; double-click and it opens the full screen. Below, a donut shows where the risk sits by lane, a cumulative recovery trend, the four lanes, a 'top opportunities' list, and a live agent-activity feed."

3. Cases and the decision log

[Go to Cases, open one case]

"Every at-risk event becomes a case. The table shows the customer, lane, amount, the VERIFIED current state, the proposed action, and a confidence score. I can filter, sort, and bulk-action."

"Open one and you get the decision log - the whole story top to bottom: the original event, the reconcile result, the classification and reason code, the chosen action, and the outcome. This is the audit trail as a screen. If a case was an intelligent no-op, it says so and explains why."

4. Guardrails - safety you can see

[Go to Guardrails]

"Safety isn't a claim here, it's a screen. This is 'What the agent refused to do.'"

"Each card is a real refusal: a message blocked because there was no consent; a manipulative draft rejected by the dark-pattern screen; a tool call blocked because it wasn't on that agent's allowlist. Nothing is silently dropped - every refusal is logged with the exact rule that stopped it."

5. Ledger - provable money

[Go to Ledger, click Simulate refund]

"All money lives in one append-only ledger. Recoveries are positive entries; refunds are negative. Nothing is ever edited or deleted."

"Watch - I'll simulate a refund landing on a recovered case. A negative entry appends, gross stays the same, and NET auto-corrects. That's why a recovery can never be overstated."

6. Replay - the proof

[\[Go to Replay, press Run Replay \]](#)

"This is the centerpiece. Replay streams six scripted moments through the SAME services production uses."

"One - a stale 'failed' webhook on an already-captured payment becomes a no-op. Two - a duplicate and out-of-order event is blocked, not double-charged. Three - a consented cart is recovered and attributed only after a verified capture. Four - a subscription that failed all three daily retries is stopped at the budget. Five - a high-value receivable is escalated to a human. Six - a refund auto-subtracts from net."

"And this toggle recomputes INCREMENTAL recovery - treatment minus a matched holdout - so we're measuring lift, not vanity."

7. Metrics - the scoreboard

[\[Go to Metrics \]](#)

"Ten measures, honestly labelled: at-risk, eligible rate, recovery rate, gross, incremental (marked simulated), net, false-action rate, customer-harm rate, automation-vs-escalation, and audit completeness - which is 100%."

"And a one-click audit export to JSON and CSV - the receipts."

8. Agents - bounded intelligence

[\[Go to Agents \]](#)

"Seven agents. Three use the LLM - the Classifier, the Communicator, the Receivables Reader. Three are pure deterministic code - the Prioritiser, the Retry Strategist, the Escalation Builder - plus an Orchestrator."

"Every agent has ONE job, a fixed tool allowlist you can edit, and an abstention rule - if it isn't sure, it escalates instead of guessing. The AI never touches money."

9. Baymax and Settings

[\[Click the Baymax orb; then open Settings \]](#)

"Baymax is the in-app assistant - a draggable companion. Offline it uses a built-in guide; connect a Google Gemini key in Settings and it answers with full AI. Admins manage the workspace, the team, and connections here."

10. Close

[\[Return to Overview \]](#)

"So that's ReOS. The merchant gets the recovered revenue and a complete audit trail. Razorpay's upside - a stickier platform through conversion and retention - is a separate, measured hypothesis;

we never conflate the two."

"It's built on Next.js, runs fully offline with zero keys, and the credibility core is tested. Recover more revenue - and prove every rupee. Thank you."

Cheat sheet - numbers and soundbites

At-risk value	Rs 1.95 lakh
Gross recovered	Rs 38.7K
Incremental recovered (vs holdout)	Rs 35.7K
Recovery rate / Audit completeness	~38% / 100%
Lanes	4 (payments, checkout, subscriptions, receivables)
Agents	7 (3 AI + 3 deterministic + orchestrator)
Replay moments	6 scripted
Screens / Roles	10 screens / 4 roles
Runs offline	Yes - zero external keys
Repo	github.com/Kajayjohnpauldev/REOS
Demo login	<code>ajay@lumina.example</code> / 12345619

Three soundbites to land:

"We re-read reality before we act - so we never charge twice."

"AI drafts. Only the ledger moves money."

"We don't just recover - we prove it, against a holdout."